BÖLÜM 4: ASENKRON UFUKLAR — OLAY ODAKLI MIMARI

💣 9 dk • DERS 16

Parça 16: Event Sourcing ve CQRS: Geçmiş, Şimdiki Zamandan Daha Önemli Olduğunda

Mimari karmaşıklıkta boğulmadan denetime açık ve geri sarılabilir sistemler kurmak.

> 🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**
>
> Çok uluslu bir bankanın hesap yönetim sistemi bakiye bilgisini basit bir SQL satırı olarak tutuyordu: `UPDATE hesaplar SET bakiye = bakiye - 500 WHERE id = 12`. Bir sabah kurumsal bir müşteri panikle aradı: 'Dün bakiyemiz 1.2 milyon TL idi, bugün 450.000 TL görünüyor! Bu para nereye gitti?' Veritabanı yöneticisi logları aradı ama veritabanında yalnızca tek bir rakam vardı: 450.000 TL. O rakamın hangi işlemlerle, hangi otomatik kesintilerle oraya geldiğini kanıtlayan hiçbir iz yoktu! Banka denetimden geçemedi ve 250.000 TL ceza yedi. Şirket her para giriş çıkışını değiştirilemez bir olay olarak saklayan Event Sourcing mimarisine geçmek zorunda kaldı.

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

Geleneksel CRUD mimarileri tarihi yok eder: Bir satırı güncellediğinizde eski durum sonsuza dek silinir. **Event Sourcing** bunu tersine çevirir: Gerçeğin kaynağı mevcut durum değil, geçmişte yaşanan değiştirilemez olayların günlüğüdür (`HesapAcildi`, `ParaYatirildi`, `ParaCekildi`). Mevcut durum bu olayların baştan sona oynatılmasıyla hesaplanan bir sonuçtur. Bunu **CQRS (Command Query Responsibility Segregation)** ile birleştirdiğinizde; yazma işlemleri (Komutlar) olay günlüğüne giderken, okuma işlemleri (Sorgular) optimize edilmiş hazır tablolardan sunulur.

Temel Mimari İlkeler ve Zihinsel Modeller

CQRS ve Event Sourcing Bileşenleri

- **Komut Tarafı:** İş kurallarını doğrular ve olay günlüğüne (Event Store) yeni olayı ekler.
- **Olay Deposu (Event Store):** Yalnızca ekleme yapılan (append-only) veritabanı. Olaylar asla silinmez veya güncellenmez.
- **Okuma Modeli (Projeksiyonlar):** Olayları dinleyip okuma için optimize edilmiş tablolara yazan işleyiciler (Örn: Arama için Elasticsearch, bakiye sorgusu için Redis).
- **Anlık Görüntü (Snapshotting):** Mevcut durumu bulmak için 100.000 olayı baştan sona oynatmamak için her 100 olayda bir ara durum kaydı almak.

Ne Zaman Kullanılmamalı? (Pratik Uyarı)

Event Sourcing çok yüksek zihinsel yük getirir: 5 yıl önceki olay şemalarının güncellenmesi zordur, eventual consistency ekipleri zorlar ve KVKK 'Unutulma Hakkı' için kriptografik silme taklaları gerektirir. Sadece finans, sigorta ve denetim gerektiren çekirdek alanlarda kullanın; basit bir blogda asla kullanmayın.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Her Yere Event Sourcing:** Basit bir profil ayarları veya blog yazısı için Event Sourcing kurup 10 kat daha fazla kod yazmak.
- **Değiştirilen Olay Hatası:** Projeksiyon hatası yüzünden olay deposuna yazılmış geçmiş bir olayı veritabanından güncellemeye kalkışmak.

- **Eksik Snapshot:** 5 milyon olayı her istekte sıfırdan oynatmaya çalışarak API gecikmelerini 30 saniyeye fırlatmak.

> 🎯 **MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** Event Sourcing'i sadece denetim geçmişinin ürünün kendisi olduğu alanlara (finans, kargo geçmişi) sakla.
> - **2. Adım:** Olay şemalarını baştan savunmacı tasarla: Olay alanlarını versiyonla (`SiparisAlindiV1`, `SiparisAlindiV2`).
> - **3. Adım:** Durum yeniden oluşturma sürelerini 10 milisaniyenin altında tutmak için ilk günden Snapshotting mekanizmasını kur.

Mimari Çıkarımlar ve Hile Tablosu

- CRUD anlık durumu gösterir; Event Sourcing o duruma nasıl geldiğinizin hikayesini anlatır.
- CQRS yazma modeliyle okuma modelini birbirinden ayırarak ikisini de bağımsız optimize etmenizi sağlar.
- Event Sourcing bir Ferrari'dir: İnanılmaz güçlüdür ama bakımı ve yakıtı son derece pahalıdır.

> 🎙️ **UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Şirketinizde geçmiş hareketlerin silinmez bir kütükte saklanmasından en çok fayda sağlayacak alan neresidir? CQRS'i mevcut sisteminize sokmanın en büyük riski ne olurdu?